

This notebook was created by [Jean de Dieu Nyandwi](#) for the love of machine learning community. For any feedback, errors or suggestion, he can be reached on email ([johnjw7084 at gmail dot com](#)), [Twitter](#), or [LinkedIn](#).

Exploratory Data Analysis (EDA)

EDA is an important step in ML project. This is where we get to dig into the data, trying to understand more about it.

By performing EDA, we can:

- Know the summary statistics
- Spot if there are missing values and the appropriate strategy for handling it
- Spot if the data is skewed (or imbalanced)
- Correlation between features
- Understand the important features (or attributes) worth keeping and unhelpful features
- Above all, answer some of the pressing questions about the data. These questions can be specific to data and its features but will around things like why this and this, or what could have caused this and this based on the analysis, etc...There are no right or wrong questions, the idea here is to use data to see if we can help answer some questions.

Real World Data Exploratory Analysis : Titanic

In this lab, we will use Titanic dataset which is available on Kaggle. We can also load it from Seaborn datasets but we will use the real data from [Kaggle](#).

Contents:

- [1. Imports](#)
- [2. Loading the dataset](#)
- [3. Quick look into the dataset](#)
- [4. Summary statistics](#)
- [5. Basic information](#)
- [6. Missing data](#)
- [7. More analysis](#)
- [8. Checking feature correlations](#)

1. Imports

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

2. Loading the dataset

```
titanic = sns.load_dataset('titanic')  
  
# titanic = pd.read_csv('train.csv')  
  
# titanic_test = pd.read_csv('test.csv')
```

If the data was coming from external sources, say `CSV`, we could have used Pandas to read the file.

```
titanic = pd.read_csv('.....')
```

3. Quick Look into the dataset

```
titanic.info()  
  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 15 columns):  
#   Column             Non-Null Count  Dtype  
---  -  
0   survived            891 non-null    int64  
1   pclass              891 non-null    int64  
2   sex                  891 non-null    object  
3   age                  714 non-null    float64  
4   sibsp               891 non-null    int64  
5   parch               891 non-null    int64  
6   fare                891 non-null    float64  
7   embarked            889 non-null    object  
8   class                891 non-null    category  
9   who                  891 non-null    object  
10  adult_male           891 non-null    bool  
11  deck                 203 non-null    category  
12  embark_town          889 non-null    object  
13  alive                891 non-null    object  
14  alone                891 non-null    bool  
dtypes: bool(2), category(2), float64(2), int64(4), object(5)  
memory usage: 80.7+ KB
```

By default, `titanic.head()` will display the first five rows. But you can display the rows you want, just pass the integer. Example: `titanic.head(10)`..`tail()` will display the last rows

```
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \ 0	0	3	male	22.0	1	0	7.2500	S
Third								
1	1	1	female	38.0	1	0	71.2833	C
First								
2	1	3	female	26.0	0	0	7.9250	S
Third								
3	1	1	female	35.0	1	0	53.1000	S
First								
4	0	3	male	35.0	0	0	8.0500	S
Third								

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

titanic.tail()

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \ 886	0	2	male	27.0	0	0	13.00	S
Second								
887	1	1	female	19.0	0	0	30.00	S
First								
888	0	3	female	NaN	1	2	23.45	S
Third								
889	1	1	male	26.0	0	0	30.00	C
First								
890	0	3	male	32.0	0	0	7.75	Q
Third								

	who	adult_male	deck	embark_town	alive	alone
886	man	True	NaN	Southampton	no	True
887	woman	False	B	Southampton	yes	True
888	woman	False	NaN	Southampton	no	False
889	man	True	C	Cherbourg	yes	True
890	man	True	NaN	Queenstown	no	True

4. Summary statistics

titanic.describe()

	survived	pclass	age	sibsp	parch
fare					

```

count  891.000000  891.000000  714.000000  891.000000  891.000000
891.000000
mean    0.383838    2.308642    29.699118    0.523008    0.381594
32.204208
std     0.486592    0.836071    14.526497    1.102743    0.806057
49.693429
min     0.000000    1.000000    0.420000    0.000000    0.000000
0.000000
25%     0.000000    2.000000    20.125000    0.000000    0.000000
7.910400
50%     0.000000    3.000000    28.000000    0.000000    0.000000
14.454200
75%     1.000000    3.000000    38.000000    1.000000    0.000000
31.000000
max     1.000000    3.000000    80.000000    8.000000    6.000000
512.329200

```

```
titanic.describe().transpose()
```

	count	mean	std	min	25%	50%	75%
max							
survived	891.0	0.383838	0.486592	0.00	0.0000	0.0000	1.0
1.0000							
pclass	891.0	2.308642	0.836071	1.00	2.0000	3.0000	3.0
3.0000							
age	714.0	29.699118	14.526497	0.42	20.1250	28.0000	38.0
80.0000							
sibsp	891.0	0.523008	1.102743	0.00	0.0000	0.0000	1.0
8.0000							
parch	891.0	0.381594	0.806057	0.00	0.0000	0.0000	0.0
6.0000							
fare	891.0	32.204208	49.693429	0.00	7.9104	14.4542	31.0
512.3292							

5. Basic Information

1. How many people who survived and died from the titanic crash? Can you use the appropriate visualization to show these people?

```

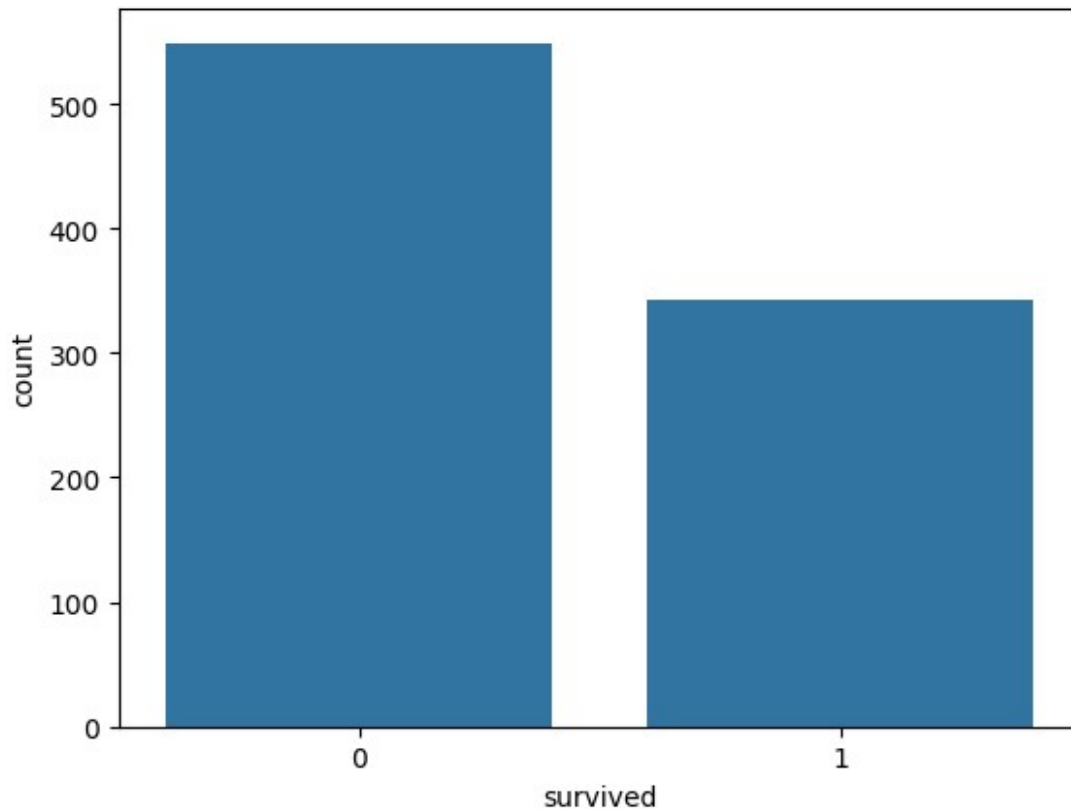
titanic['survived'].value_counts()

survived
0      549
1      342
Name: count, dtype: int64

sns.countplot(data=titanic, x='survived')

<Axes: xlabel='survived', ylabel='count'>

```



1. How many pclass are there in the dataset? Is it the same as the text column class?

```
titanic['pclass'].value_counts()

pclass
3    491
1    216
2    184
Name: count, dtype: int64

titanic['class'].value_counts()

class
Third    491
First    216
Second   184
Name: count, dtype: int64

p_class = titanic[['pclass', 'class']]
p_class.head()

   pclass  class
0       3  Third
1       1  First
2       3  Third
```

```
3      1  First
4      3  Third
```

It seems that both of these columns are the same except that one is numeric and other is text. They basically contain the same information of the class that the passengers paid for.

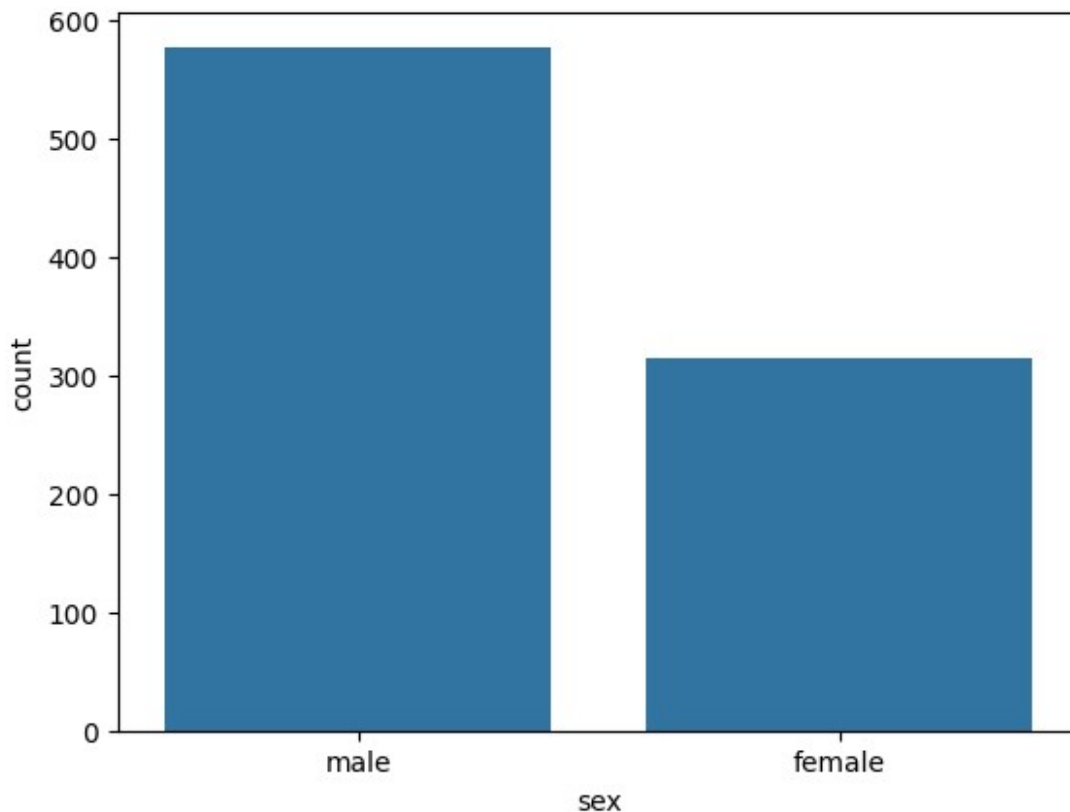
1. How many genders are there in the dataset?

```
titanic['sex'].value_counts()

sex
male      577
female    314
Name: count, dtype: int64

sns.countplot(data=titanic, x='sex')

<Axes: xlabel='sex', ylabel='count'>
```



1. Show the passengers who were less than 20 years

```
less_than_20 = titanic[titanic['age'] < 20 ]
less_than_20.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \								
7	0	3	male	2.0	3	1	21.0750	S
Third								
9	1	2	female	14.0	1	0	30.0708	C
Second								
10	1	3	female	4.0	1	1	16.7000	S
Third								
14	0	3	female	14.0	0	0	7.8542	S
Third								
16	0	3	male	2.0	4	1	29.1250	Q
Third								

	who	adult_male	deck	embark_town	alive	alone
7	child	False	NaN	Southampton	no	False
9	child	False	NaN	Cherbourg	yes	False
10	child	False	G	Southampton	yes	False
14	child	False	NaN	Southampton	no	True
16	child	False	NaN	Queenstown	no	False

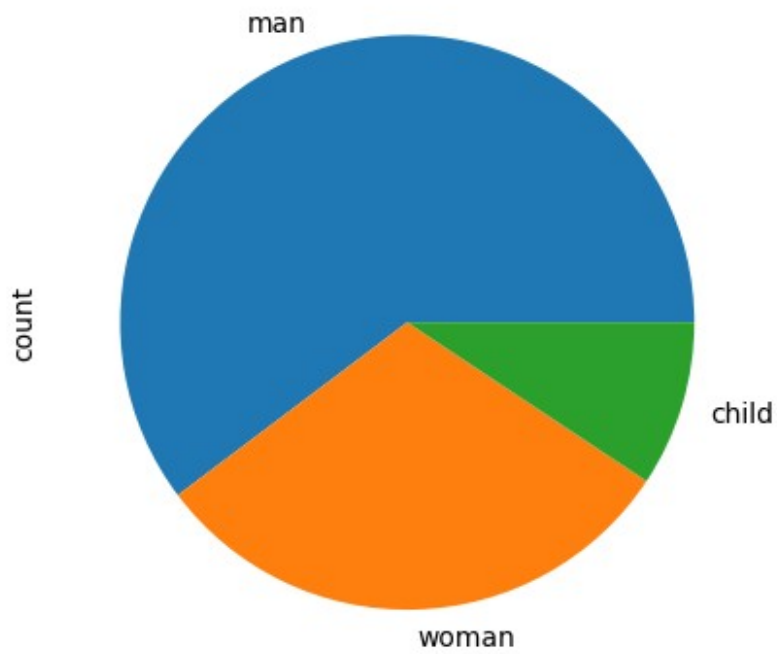
```
len(less_than_20)
```

```
164
```

1. Show the categories of the passengers in the column `who`. Use Pie chart to illustrate their values

```
titanic.who.value_counts().plot(kind='pie')
```

```
<Axes: ylabel='count'>
```



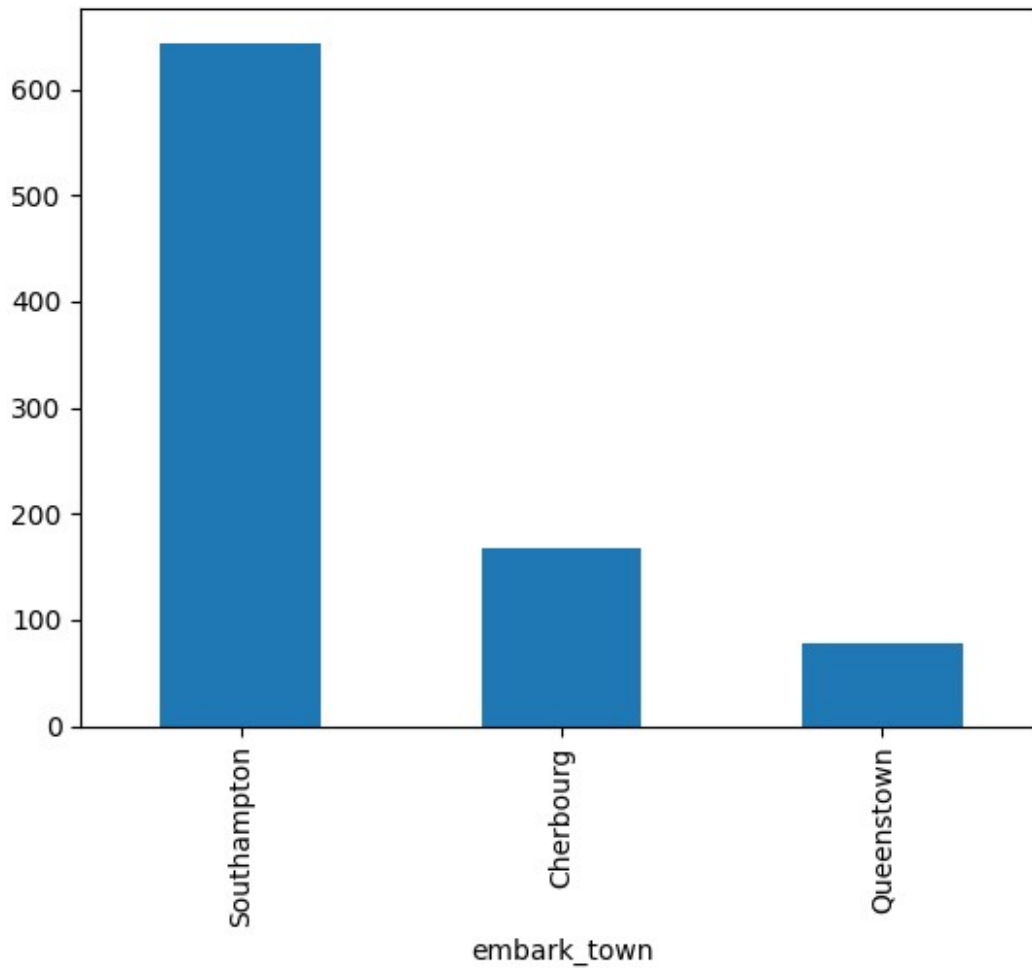
1. How many unique cities are there in the column `embark_town`? Plot their occurrences.

```
titanic['embark_town'].nunique()
```

```
3
```

```
titanic['embark_town'].value_counts().plot(kind='bar')
```

```
<Axes: xlabel='embark_town'>
```

There is no limit to how deep you can go to understand the dataset.

6. Missing Data

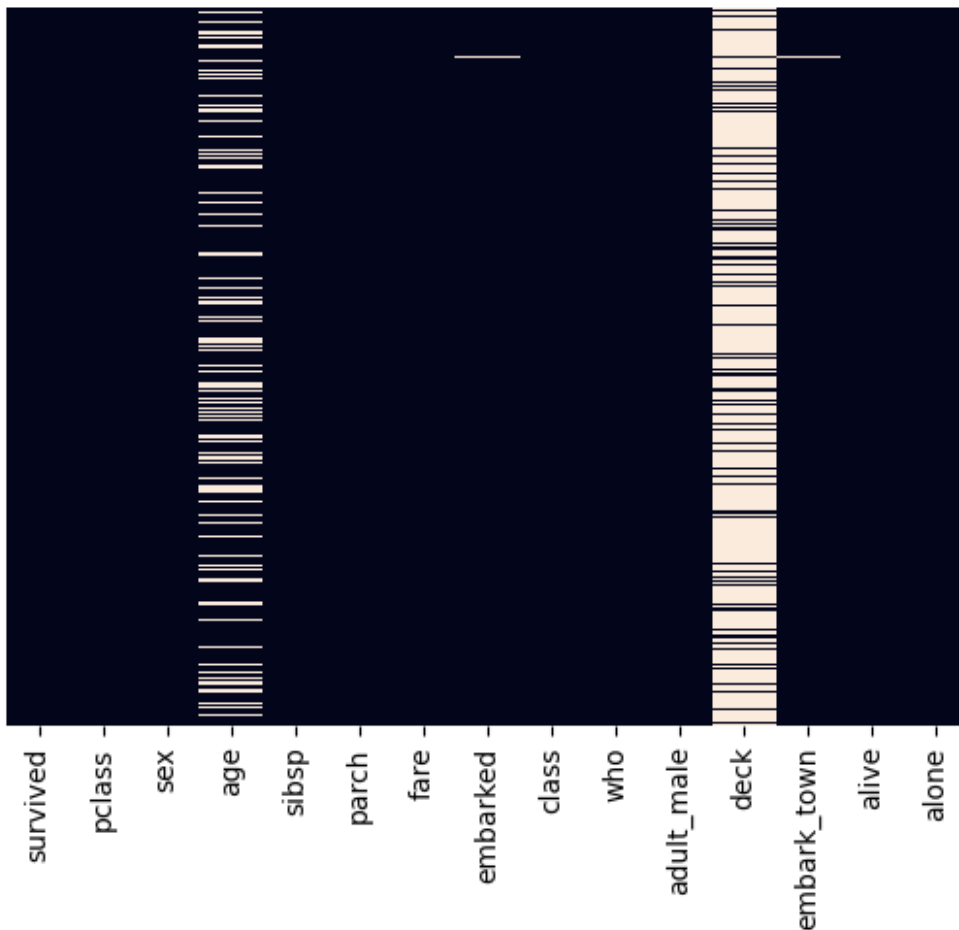
```
titanic.isnull().sum()
```

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688

```
embark_town    2
alive          0
alone          0
dtype: int64
```

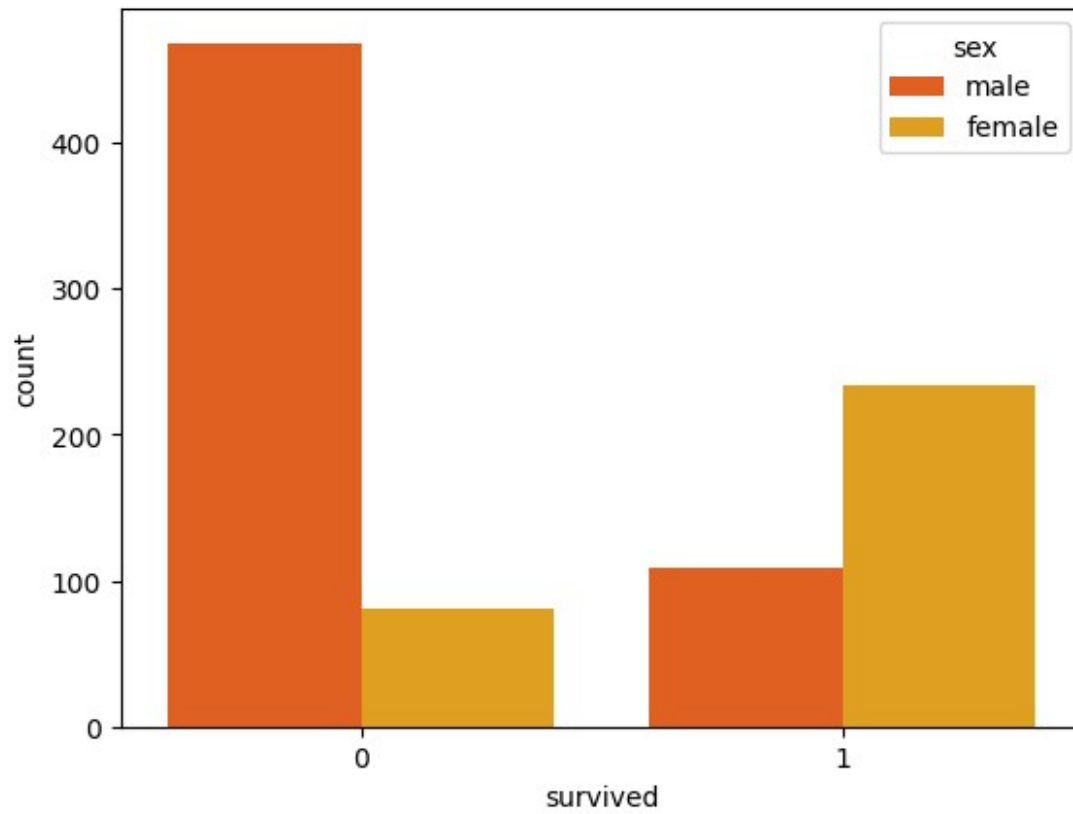
There are missing data in age and deck columns. We can also use heatmaps to show the missing values

```
sns.heatmap(titanic.isnull(), yticklabels=False, cbar=False)
<Axes: >
```



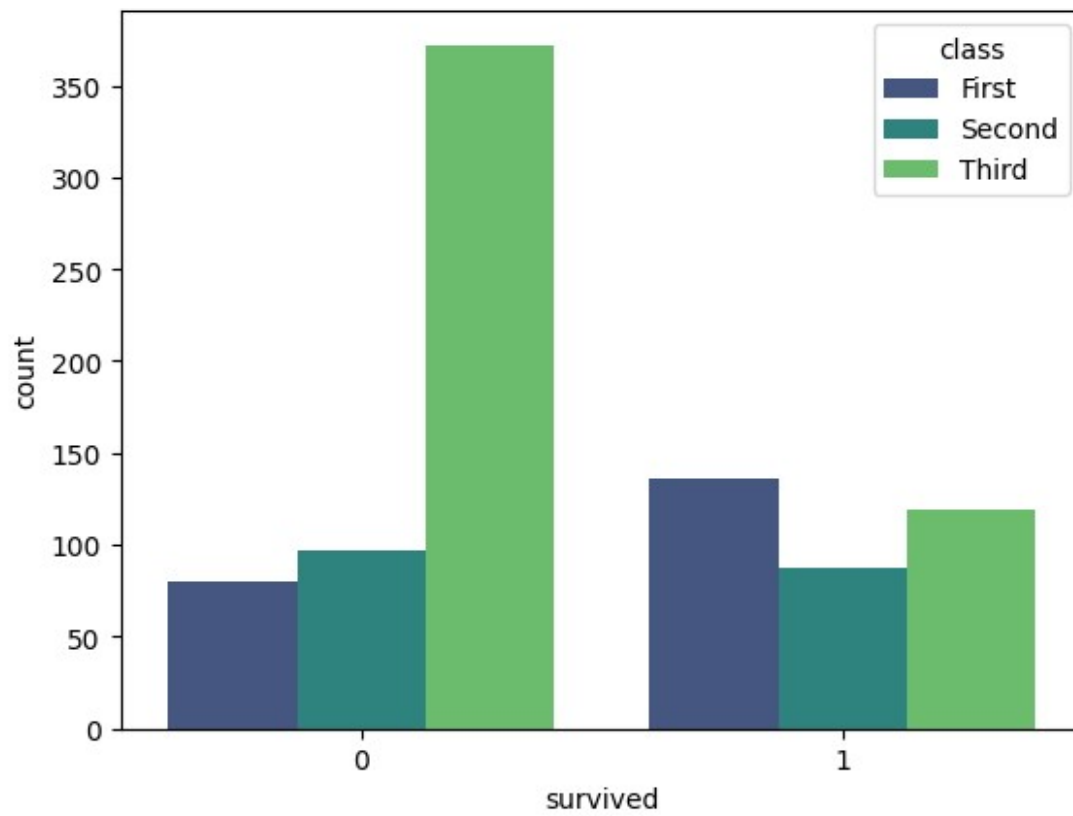
7. More Analysis

```
sns.countplot(data=titanic, x='survived', palette='autumn', hue='sex')
<Axes: xlabel='survived', ylabel='count'>
```

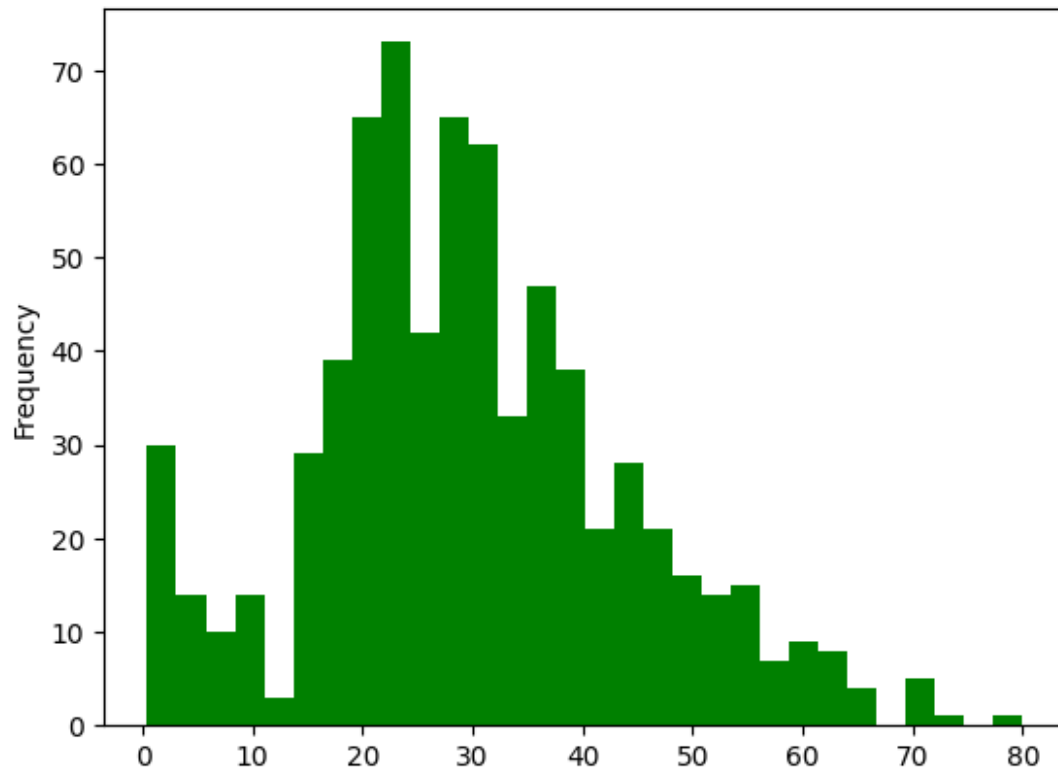


```
sns.countplot(data=titanic, x='survived', palette='viridis',  
hue='class')
```

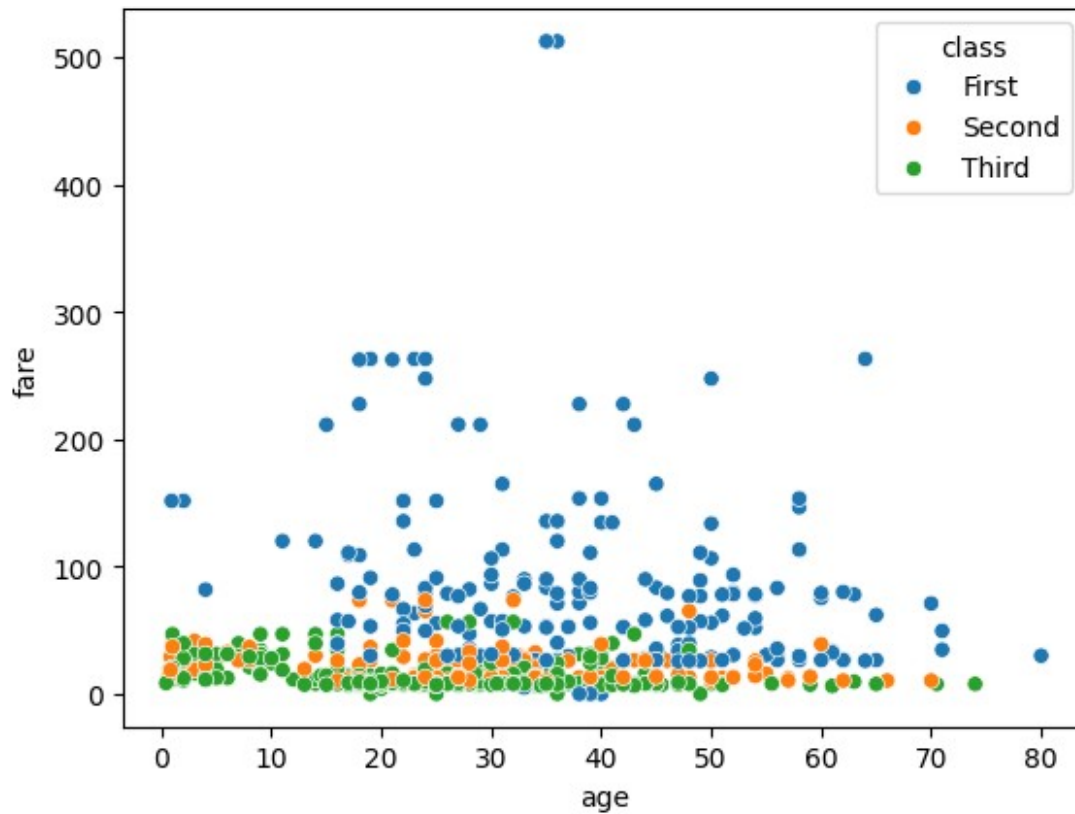
```
<Axes: xlabel='survived', ylabel='count'>
```



```
titanic.age.plot(kind='hist',bins=30,color='green')  
<Axes: ylabel='Frequency'>
```

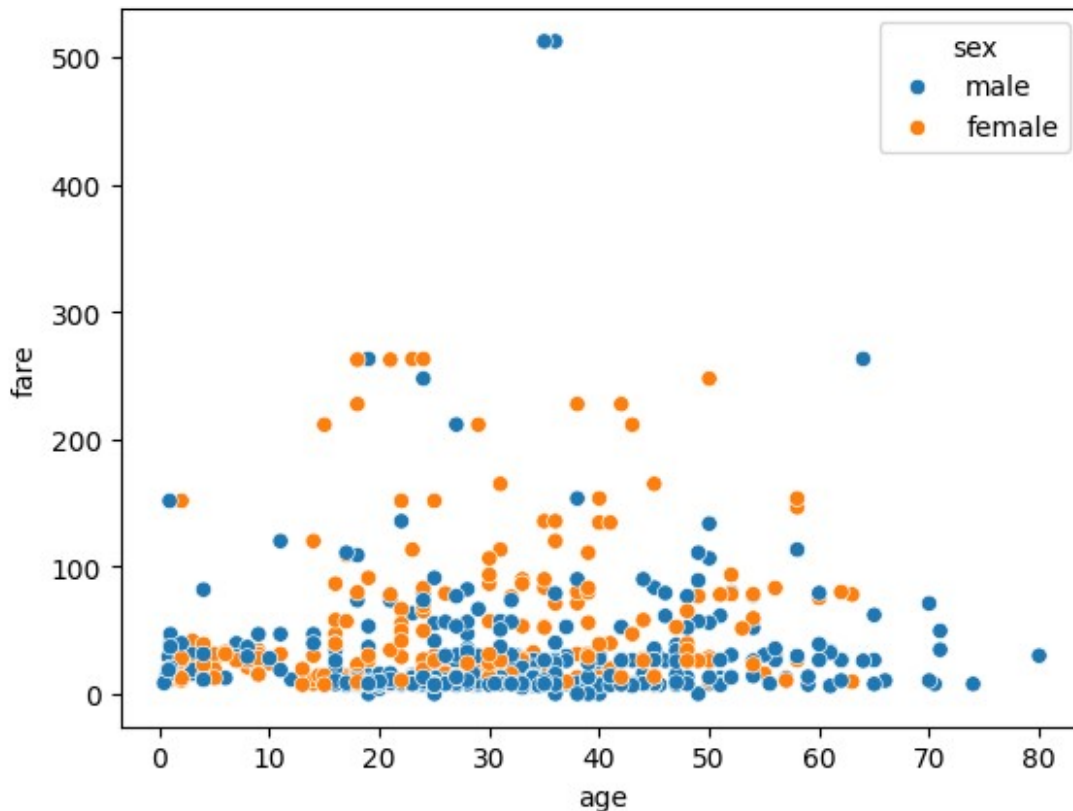


```
sns.scatterplot(data=titanic, x='age', y='fare', hue='class')  
<Axes: xlabel='age', ylabel='fare'>
```



That is obvious. The passengers who were in the first class paid more than other classes. And many females were in the first class.

```
sns.scatterplot(data=titanic, x='age', y='fare', hue='sex')  
<Axes: xlabel='age', ylabel='fare'>
```



The purpose of this lab was to learn about the exploratory analysis. There is no limit to what you can do. The more time you spend with the data, the good your analysis will be. I used questions to make it simple and lead the way but there is no proper format for this. The goal is to be one with data (CC: Andrej).

8. Checking Correlating Features

Checking correlation can help you to see the similarity between features. If two features correlate, that means they contain the same information and if one of them is removed, the analysis/model can be less affected.

```
correlation = titanic.corr(numeric_only=True)
correlation
```

	survived	pclass	age	sibsp	parch	fare
survived	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
pclass	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
age	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
sibsp	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651

parch	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
fare	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000
adult_male	-0.557080	0.094035	0.280328	-0.253586	-0.349943	-0.182024
alone	-0.203367	0.135207	0.198270	-0.584471	-0.583398	-0.271832

	adult_male	alone
survived	-0.557080	-0.203367
pclass	0.094035	0.135207
age	0.280328	0.198270
sibsp	-0.253586	-0.584471
parch	-0.349943	-0.583398
fare	-0.182024	-0.271832
adult_male	1.000000	0.404744
alone	0.404744	1.000000

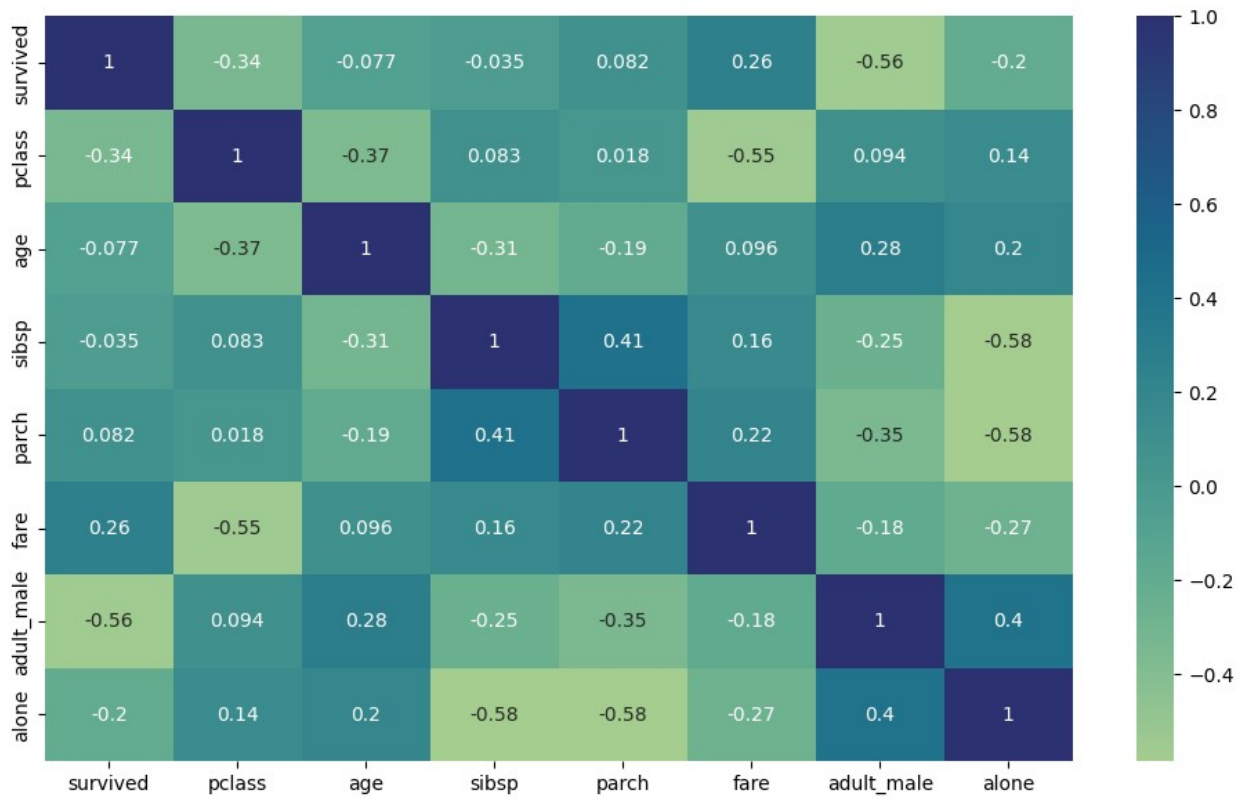
```
correlation['survived']
```

```
survived    1.000000
pclass      -0.338481
age         -0.077221
sibsp       -0.035322
parch        0.081629
fare         0.257307
adult_male  -0.557080
alone       -0.203367
Name: survived, dtype: float64
```

If you want to check how the features correlate with the label (`survived`, just for example), here is how to do it:

```
#### Visualizing correlation
```

```
plt.figure(figsize=(12,7))
sns.heatmap(correlation,annot=True,cmap='crest')
<Axes: >
```

This is the end of the lab!

[Back to top](#)

